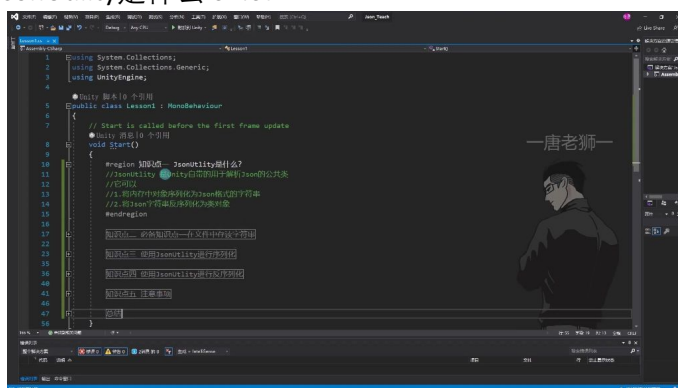


一、数据持久化 00:04

1. JsonUtility序列化 00:11

1) 课程要点

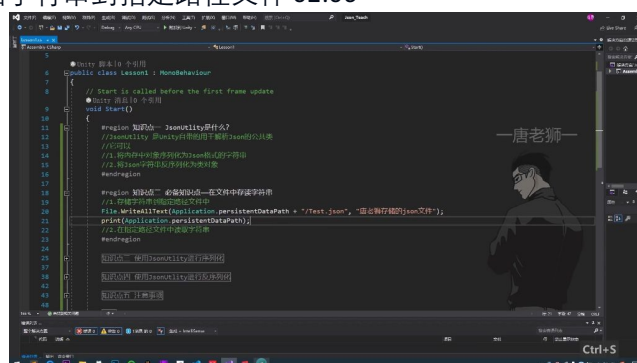
2) JsonUtility是什么 01:07



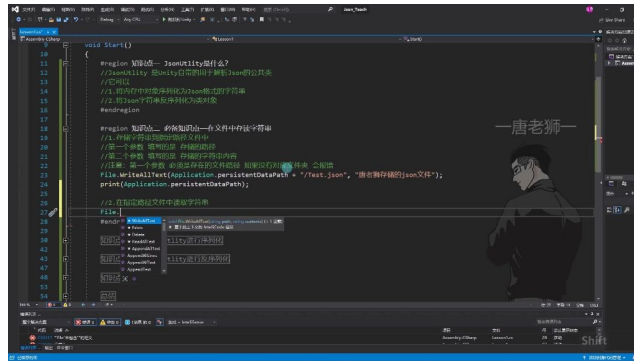
-
- 主要作用: Unity自带的用于解析Json的公共类
- 核心功能:
 - 将内存中对象序列化为Json格式的字符串
 - 将Json字符串反序列化为类对象

3) 必备知识点 01:38

- 在文件中存读字符串 01:44
 - 前置知识: 需要引用System.IO命名空间
 - 关键类: File类提供相关API
- 存储字符串到指定路径文件 02:06



-
- API: File.WriteAllText()
- 参数说明:
 - 第一个参数: 存储路径 (必须确保路径文件夹存在)
 - 第二个参数: 要存储的字符串内容
- 示例代码:
- 注意事项:
 - 使用Application.persistentDataPath作为可读写路径
 - 路径中不能包含不存在的文件夹, 否则会报错
 - 可以直接写文件名和后缀, 不用自定义文件夹路径
- 在指定路径文件中读取字符串 06:56

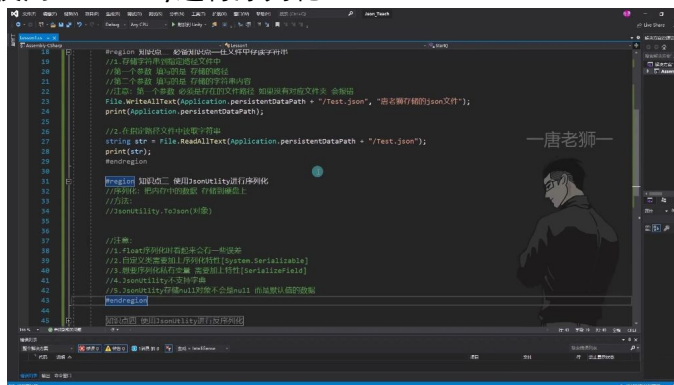


-
- **API: File.ReadAllText()**
- **参数说明:**
 - 唯一参数: 要读取的文件路径
- **返回值:** string类型, 返回文件内容
- **示例代码:**
- **实际应用:**
 - 读取后会返回之前存储的完整字符串内容
 - 可用于后续的Json反序列化操作

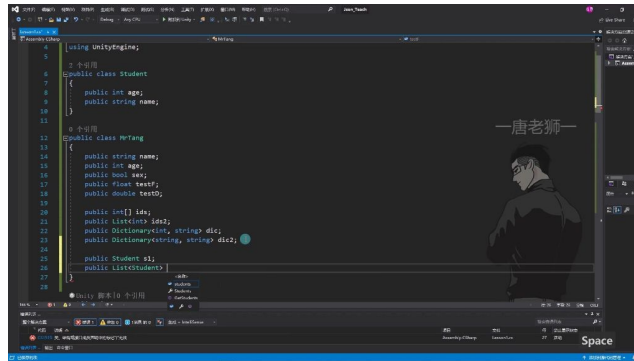
4) 使用JsonUtility进行序列化

- **API: JsonUtility.ToJson()**
- **注意事项:**
 - 自定义类需要添加[System.Serializable]特性
 - 要序列化私有变量需要添加[SerializeField]特性
 - JsonUtility存储null对象时会转为默认值

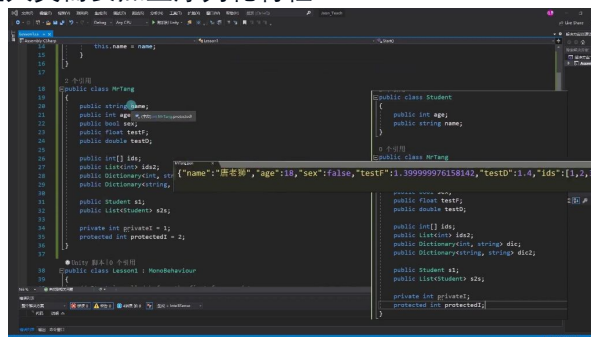
5) 使用JsonUtility进行序列化 08:10



-
- **基本概念:** 序列化是将内存中的数据存储到硬盘上的过程, 使用JsonUtility.ToJson(对象)方法可将对象转换为JSON格式字符串
- **存储方法:** 通过File.WriteAllText将JSON字符串写入指定路径文件, 路径参数必须存在否则会报错
- **读取方法:** 使用File.ReadAllText从指定路径读取JSON字符串内容
- **举例学生类进行序列化 08:42**



- 类结构示例:
 - 基础类型: string name、int age、bool sex、float testF、double testD
 - 集合类型: int[] ids、List<int> ids2
 - 复杂类型: Dictionary<int,string> dic、Student s1、List<Student> s2s
 - 特殊成员: private int privatel、protected int protectedl
- 序列化过程:
 - 创建MrTang对象并初始化所有字段
 - 使用JsonUtility.ToJson(t)生成JSON字符串
 - 通过File.WriteAllText存储到Application.persistentDataPath路径
- 序列化注意事项 17:27
 - float序列化时有误差
 - 现象: float类型序列化后可能出现精度误差 (如1.4变为1.399999976158142)
 - 原因: JSON序列化过程中的浮点数转换问题
 - 影响: 仅视觉差异, 实际反序列化后数值仍正确
 - 自定义类需要加上序列化特性 18:01



- 要求: 被嵌套的自定义类必须添加[System.Serializable]特性
- 示例:

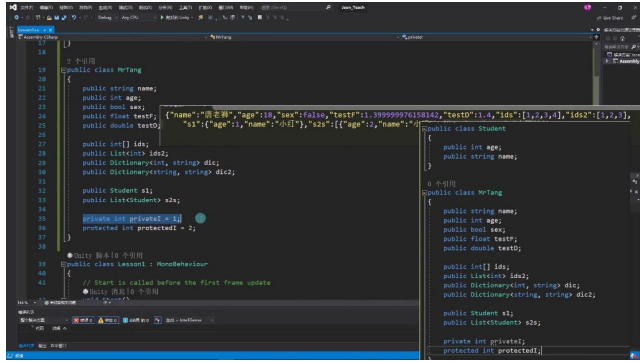
```

1 [System.Serializable]
2 public class Student {
3     public int age;
4     public string name;
5 }

```

1 - **注意**: 最外层类不需要此特性, 仅内部嵌套的自定义类需要

- 想要序列化私有变量需要加上特性 20:02

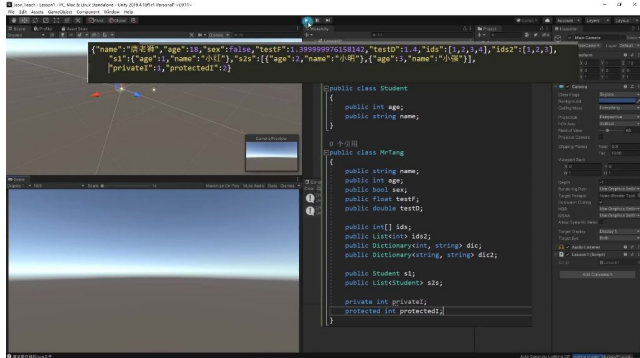


-
- 方法：私有成员需添加[SerializeField]特性
- 示例：

```
1 [SerializeField]
2 private int private1;
3 [SerializeField]
4 protected int protected1;
```

1 - **效果**：添加后私有/保护成员也能被正确序列化

- JsonUtility不支持字典 21:22
 - 限制：Dictionary类型无法被JsonUtility直接序列化
 - 表现：字典数据不会出现在生成的JSON字符串中
 - 解决方案：需要自定义字典的序列化方式或使用其他JSON库
- JsonUtility存储null对象不会是null而是默认值的数据 21:42



-
- 现象：当对象为null时，序列化结果为该类型的默认值
- 示例：
 - Student对象null会序列化为{"age":0,"name":""}
 - 不会保留真正的null值
- 影响：需要注意反序列化时的默认值处理逻辑

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
JSON Utility 简介	Unity 自带的 JSON 解析公共类，支持序列化（对象→JSON 字符串）和反序列化（JSON 字符串→对象）。	序列化与反序列化的核心作用，需明确其应用场景（如数据持久化）。	★★
文件 存读	使用 File 类的 WriteAllText（存储）和 ReadAllText	路径必须存在，否则报错；默认路径为 Application.persistentDataPath。	★★

字符串	(读取) 实现字符串的本地化存读。		
JSON 序列化	通过 <code>JsonUtility.ToJson</code> 将类对象转为 JSON 字符串，需注意： - 自定义类需加 [Serializable] 特性 - 私有变量需加 [SerializeField] 特性 - 不支持字典类型 - <code>float/double</code> 可能有精度误差 - 空对象会转为默认值。	特性遗漏导致序列化失败；字典需手动处理；数值类型精度问题需提前告知用户。	★★★
JSON 反序列化	使用 <code>JsonUtility.FromJson</code> 将 JSON 字符串还原为对象（需提前定义匹配的结构）。	类结构必须与 JSON 字段完全匹配，否则数据丢失。	★★★
注意事项总结	1. 文件路径需预先存在 2. 序列化特性不可遗漏 3. 字典和 <code>long</code> 类型需特殊处理 4. 浮点数精度误差属正常现象。	易忽略点：特性作用域（外层类无需 [Serializable] ，内嵌类需要）。	★★